

파이썬 가상환경(venv)

Python Virtual Environment

왜 써야 할까? 그리고 어떻게 만들까?



01. 가상환경이란?

파이썬 패키지를 독립적으로 관리하여
프로젝트별 최적의 환경을 유지하는 도구

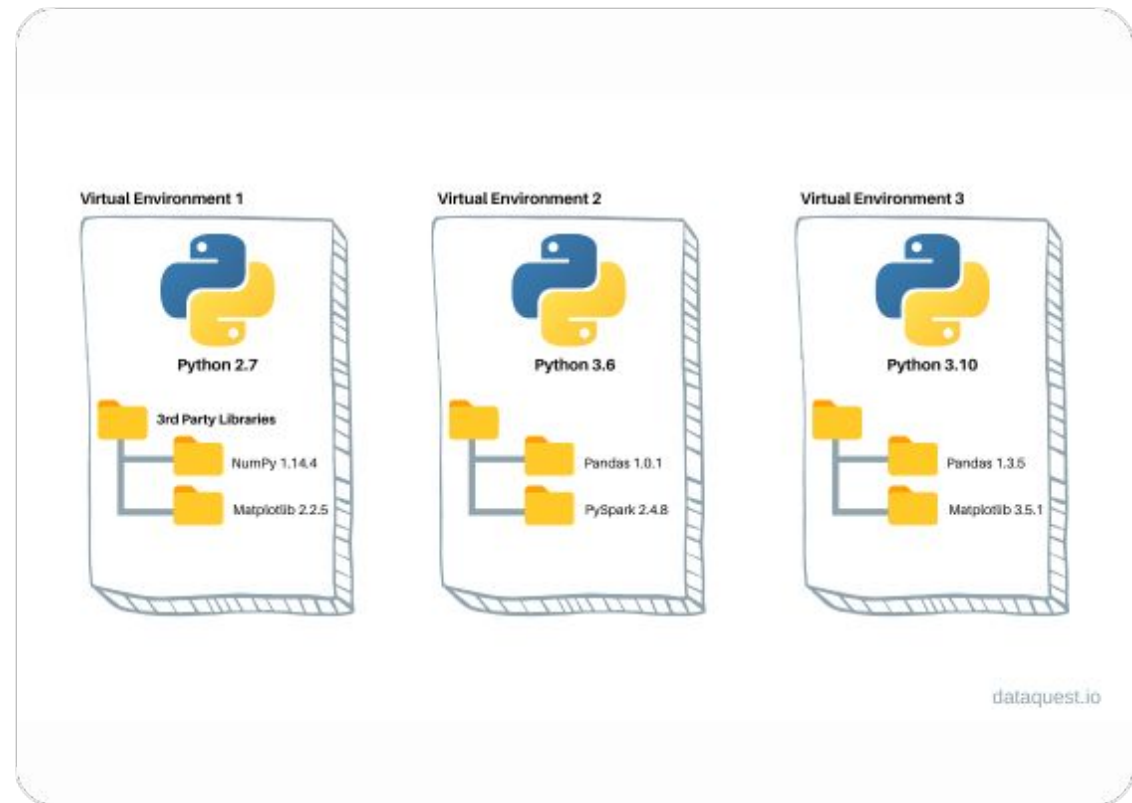
| 독립적인 환경의 필요성

프로젝트 A: Django 3.2 사용

프로젝트 B: Django 4.0 사용

시스템 기본 패키지와 충돌 없이 독립적으로 유지

✓ 같은 컴퓨터에서 여러 버전 동시 운영 가능



| 가상환경이 필요한 3가지 이유



버전 격리

프로젝트마다 필요한
라이브러리 버전을 다르게
설정할 수 있습니다.



충돌 방지

패키지 간의 버전 의존성 충돌을
막아 안정적인 개발이
가능합니다.



협업 효율성

모든 팀원이 동일한 패키지
환경에서 작업할 수 있도록
도와줍니다.

02. 가상환경 만들기



| Step 1: 가상환경 생성

01

venv 모듈 사용하기

현재 폴더에 myenv라는 이름의 가상환경을 생성합니다.

Terminal / Command Prompt

```
python -m venv myenv
```

 'myenv' 자리에 원하는 폴더 이름을 넣을 수 있습니다.

| Step 2: 가상환경 활성화

Mac / Linux

터미널에서 아래 명령어를 입력하세요.

```
source myenv/bin/activate
```

Windows

CMD 또는 PowerShell에서 입력하세요.

```
myenv\Scripts\activate
```

활성화 완료 시 프롬프트 앞에 (myenv) 표시가 나타납니다!

| Step 3: 비활성화 및 종료

작업을 마치고 기본 시스템 환경으로 돌아가고 싶을 때는 매우 간단합니다.

deactivate

- 🔌 별도의 경로 입력 없이 어디서든 작동
- 🔄 즉시 원래의 파이썬 환경으로 복귀



| pip (venv) vs Conda 비교

비교 항목	pip + venv	Conda
설치 기본값	Python 3.3+ 기본 포함	별도 설치 (Anaconda 등)
패키지 관리	PyPI (pip install)	Conda 전용 패키지
속도	빠름	상대적으로 느림
주요 분야	일반 웹 개발, 스크립트	데이터 과학, 머신러닝

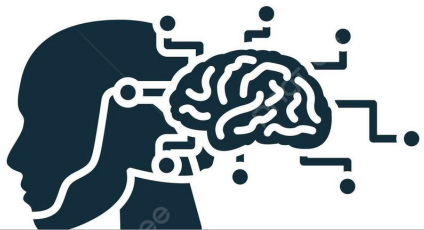
| 어떤 것을 선택할까?

프로젝트 성격에 따른 추천

웹 개발, 일반 프로젝트: 가볍고 표준적인 venv + pip 조합을 추천합니다.

데이터 분석, AI: 복잡한 라이브러리 관리가 쉬운 conda 사용이 유리합니다.

보통은 venv를 기본으로 사용하다가, 필요시 conda로 확장하면 좋습니다.



| 핵심 요약

- ✓ 가상환경(venv): 프로젝트별로 독립된 패키지 저장소
- ✓ 생성: `python -m venv 이름` 명령어로 간단하게 생성
- ✓ 활성화: OS에 맞는 activate 스크립트 실행 (프롬프트 확인)
- ✓ 용도: 일반 개발은 venv, 데이터 과학은 conda가 유리

| 가상 환경 계획

1. 기존 파이썬 수업에 사용한 **Global** 파이썬 개발환경 3.14 버전 [python.org 에서 설치한 개발환경]
2. Python 리마인드 수업 및 가상환경 실습용 가상환경 [00_Basic - venv]
3. RPA 실습에 사용할 가상환경 [01_RPA - venv]
4. Web data , 생성형 AI API, 랭체인, RAG 실습에 사용할 가상환경 [02_WebData - venv]
5. 데이터분석에 사용할 아나콘다 가상환경 [03_DataAnalysis - Conda]
6. 머신러닝에 사용할 Google Colab 가상환경 [04_ML_DL - colab]

| 프로젝트 별 가상환경 만들기 실습

1. 프로젝트 폴더 먼저 만들기

```
mkdir my_project  
cd my_project
```

2. 가상환경 생성

```
python -m venv venv  
[ -m 옵션 : module - venv 모듈을 실행]
```

3. 가상환경 활성화

macOS / Linux

```
source venv/bin/activate
```

Windows

```
venv\Scripts\activate
```

4. 이제 현재 폴더(my_project)에서 코드 작성

```
my_project/  
├─ venv/  
├─ app.py ← 여기서 개발
```

| 파이썬 인터프리터 지정

VSCODE 에서 터미널을 실행할 **PYTHON Interpreter** 선택

ctrl + shift + p (palette) [상단메뉴 View > Command Palette...]

> Python:Select Interpreter

[항목 중 경로를 확인하여 본인이 사용할 가상환경 선택]

| 가상환경 종료 및 삭제

가상 환경 종료

`deactivate`

가상 환경 삭제

(Windows)

`rmdir /s /q venv`

/s : subdirectory - 하위폴더까지 모두 삭제

/q : quiet - y/n 물어보지 말고 삭제

(macOS 및 Linux)

`rm -rf venv`

-r : recursive : 하위폴더까지 모두 삭제

-f : force : root권한이 없어서 삭제 권한이 없어도 y/n 묻지 않고 삭제

| 가상 환경을 포함하는 프로젝트 폴더 구조

my_project/

|

|─ venv/ ← 가상환경 (이름은 자유)

|─ src/ ← 실제 파이썬 소스 코드

| └─ main.py

|─ requirements.txt ← 패키지 목록

└─ README.md

venv/ 폴더는 오직 가상환경용 . 여기는 개발자가 직접 코드 넣고 작업하는 공간이 아님

간단하게 .

my_project/

|

|─ venv/

|─ main.py

└─ requirements.txt

| 프로젝트의 가상환경 패키지 목록 관리

requirements.txt 파일 생성과 설치

생성

pip freeze 명령어를 사용해 현재 가상환경에 설치된 모든 패키지와 버전을 **requirements.txt** 파일로 저장

```
pip freeze > requirements.txt
```

설치

pip install -r requirements.txt 명령어를 실행하면 동일한 패키지 환경을 다른 환경에서도 그대로 재현

```
pip install -r requirements.txt
```

| Visual Studio Code 터미널에서의 권한 문제

```
PS C:\YJ\soma\data-pipeline\pct> env\Scripts\activate
env\Scripts\activate : 이 시스템에서 스크립트를 실행할 수 없으므로 C:\YJ\soma\data-pipeline\pct\env\Scripts\Activate.ps1 파일을 로드할 수
없습니다. 자세한 내용은 about_Execution_Policies(https://go.microsoft.com/fwlink/?LinkID=135170)를 참조하십시오.
위치 줄:1 문자:1
+ env\Scripts\activate
+ ~~~~~
+ CategoryInfo          : 보안 오류: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

on PowerShell with admin [1번만 하면 됨]

Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

on VSCode [매번 해야함.]

Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned